一、ORM

二、教学目标

1、掌握模板继承使用

2、掌握模板导入使用

3、了解字段介绍等等的介绍

4、完成ORM的单表操作

5、掌握应用中的测试环境

三、模版继承

3.1 概述

当网站的多个页面的样式几乎一样,只是在局部做一些改变,将整体做成一个模版页面,

其他的页面继承这个模版页面,然后重写不同的区域

先创建一个想要继承的模版页面,在模版页面中提前划分好需要修改的区域

语法:

```
继承
{% extends 名称 %}

应用
{% block 名称 %}
      模版页面的内容
{% endblock %}
```

1 在app01应用->urls.py 创建路由

```
#搭建页面
    path('home/',views.home),
    path('login/',views.login),
    path('register/',views.register),
```

2 在app01应用-> views.py中 创建类和方法

```
#主页
def home(request):

    return render(request,'home.html')
#登录
def login(request):
    return render(request, 'login.html')

#登录
def register(request):
    return render(request, 'register.html')
```

3 在templates 创建home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    {% load static %}  <!-- 添加此行 -->

    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static '/bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "/js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "/bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<nav class="navbar navbar-default">
  <div class="container-fluid">
    <!-- Brand and toggle get grouped for better mobile display -->
    <div class="navbar-header">
      <button type="button" class="navbar-toggle collapsed" data-
toggle="collapse" data-target="#bs-example-navbar-collapse-1" aria-
expanded="false">
        <span class="sr-only">Toggle navigation</span>
        <span class="icon-bar"></span>
        <span class="icon-bar"></span>
        <span class="icon-bar"></span>
      </button>
      <a class="navbar-brand" href="#">Brand</a>
    </div>

    <!-- Collect the nav links, forms, and other content for toggling -->
    <div class="collapse navbar-collapse" id="bs-example-navbar-collapse-1">
      <ul class="nav navbar-nav">
        <li class="active"><a href="#">Link <span class="sr-only">(current)
</span></a></li>
        <li><a href="#">Link</a></li>
        <li class="dropdown">
```

```
          <a href="#" class="dropdown-toggle" data-toggle="dropdown"
role="button" aria-haspopup="true" aria-expanded="false">Dropdown <span
class="caret"></span></a>
          <ul class="dropdown-menu">
            <li><a href="#">Action</a></li>
            <li><a href="#">Another action</a></li>
            <li><a href="#">Something else here</a></li>
            <li role="separator" class="divider"></li>
            <li><a href="#">Separated link</a></li>
            <li role="separator" class="divider"></li>
            <li><a href="#">One more separated link</a></li>
          </ul>
        </li>
      </ul>
      <form class="navbar-form navbar-left">
        <div class="form-group">
          <input type="text" class="form-control" placeholder="Search">
        </div>
        <button type="submit" class="btn btn-default">Submit</button>
      </form>
      <ul class="nav navbar-nav navbar-right">
        <li><a href="#">Link</a></li>
        <li class="dropdown">
          <a href="#" class="dropdown-toggle" data-toggle="dropdown"
role="button" aria-haspopup="true" aria-expanded="false">Dropdown <span
class="caret"></span></a>
          <ul class="dropdown-menu">
            <li><a href="#">Action</a></li>
            <li><a href="#">Another action</a></li>
            <li><a href="#">Something else here</a></li>
            <li role="separator" class="divider"></li>
            <li><a href="#">Separated link</a></li>
          </ul>
        </li>
      </ul>
    </div><!-- /.navbar-collapse -->
  </div><!-- /.container-fluid -->
</nav>

<div class="container-fluid">
    <div class="row">
        <!--左边三份 -->
        <div class="col-md-3">
           <div class="list-group">
                           <a href="/app01/home/" class="list-group-item active">
首页</a>
                           <a href="/app01/login/" class="list-group-item">登录
</a>
                           <a href="/app01/register/" class="list-group-item">注册
</a>

                         </div>
        </div>
        <!--右边三份 -->
        <div class="col-md-9">
             {% block content %}
            <div class="jumbotron">
                 <h1>Hello, world!</h1>
```

```
                    <p>...</p>
                    <p><a class="btn btn-primary btn-lg" href="#"
role="button">Learn more</a></p>
                </div>
                {% endblock %}
            </div>
        </div>

</div>




</body>
</html>
```

4 创建login.html

```
<!-- 继承home.html  -->
{% extends 'home.html' %}

<!-- 重写block指定名称 content-->
{% block content %}
<h1>登录</h1>
<form action="" method="post">
    <p>用户名:<input type="text" name="username"></p>
    <p>密码:<input type="password" name="password"></p>
    <input type="submit">

</form>
{% endblock %}
```

5 创建register.html

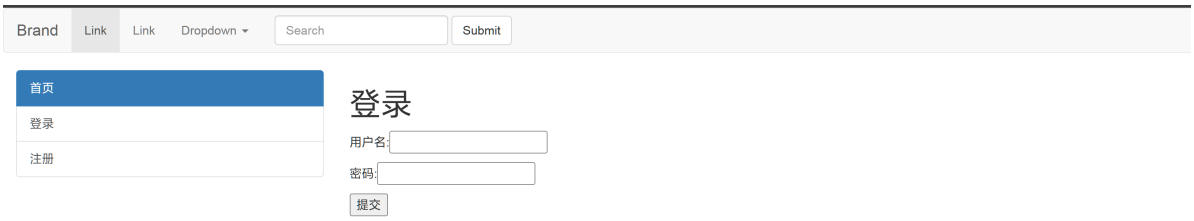
```
<!-- 继承home.html  -->
{% extends 'home.html' %}


<!-- 重写block指定名称 content-->
{% block content %}
<h1>注册</h1>
<form action="" method="post">
    <p>用户名:<input type="text" name="username"></p>
    <p>密码:<input type="password" name="password"></p>
     <p>确认密码:<input type="re_password" name="re_password"></p>
    <input type="submit">

</form>
{% endblock %}
```

3.2 完成测试

地址栏输入 http://127.0.0.1:8000/app01/home/



- 一个html页面至少可以重写3块区域
  - css区域
  - html区域
  - js区域

注意: 模版继承不能继承后端传递过来的数据

如:

我在主页把username带过去 看看其他继承的页面是否可以访问到 答案:是不能访问到的

```
#主页
def home(request):
    data = {'username':'admin'}
    return render(request,'home.html',locals())
```

home.html

```
<a class="navbar-brand" href="#">{{ data.username }}</a>
```

测试发现其他继承页面是不能继承数据

3.3 总结:

先创建一个想要继承的模版页面,在模版页面中提前划分好需要修改的区域

四、模版导入

4.1 概述

将页面的做成一个局部模版的样式,哪个地方需要就可以直接导入引用

语法:

```
{% include 你需要导入的模版页面 %}
```

语法:

4.2 代码实现

1 写一个页面作为你的模版页面 templates -> foot.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    {% load static %}
    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static '/bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "/js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "/bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<nav aria-label="Page navigation">
  <ul class="pagination">
    <li>
      <a href="#" aria-label="Previous">
        <span aria-hidden="true">&laquo;</span>
      </a>
    </li>
    <li><a href="#">1</a></li>
    <li><a href="#">2</a></li>
    <li><a href="#">3</a></li>
    <li><a href="#">4</a></li>
    <li><a href="#">5</a></li>
    <li>
      <a href="#" aria-label="Next">
```

```
        <span aria-hidden="true">&raquo;</span>
      </a>
    </li>
  </ul>
</nav>
</body>
</html>
```

2 在任意的地方都可以应用

例如: 在home.html中应用

```
{% include 'foot.html' %}
```

五、ORM

5.1 单表操作

5.1.1 配置数据库

在 `setttings.py` 中的 `DATABASES` 进行配置，代码如下:

```
DATABASES = {
    'default':{
        # 要使用的数据库后端
        'ENGINE':'django.db.backends.mysql',
        'HOST':'数据库服务器的IP地址',
        'PORT':'端口号',
        'USER': '数据库管理员用户名',
        'PASSWORD':'数据库管理员密码',
        'NAME':'数据库名称',
    }
}
```

5.1.2 创建模型

`Django` 中创建模型的语法结构是:

```
from django.db import models

class User(models.Model):
    #字符串
    name = models.CharField(max_length=32)
    #整数
    age = models.IntegerField()

    '''
        DateField/TimeField/DateTimeField两个重要的参数
        auto_now_add: 在创建数据的时候,将创建时间写入,后面只要不人修改,就不会变化
        auto_now: 每当创建数据或者修改时,将当前系统时间写入字段
    '''
    #创建时间 年月日
```

```
    create_time = models.DateField(auto_now_add=True)
```

其中：

- `ModelName` 指要创建的模型类的名称（即数据表名称）
- `fieldname` 指定义的字段名称
- `FieldType` 用于定义字段的数据类型
- `options` 用于定义字段的选项，如是否允许为空、字段默认值、数据类型的长度限制、是否为主键等

5.1.3 字段介绍

5.1.3.1 字段命名限制

- 字段的名称不能是 `Python` 保留字，这会导致 `Python` 语法错误
- 字段名称不能包含连续的多个下划线，原因在于 `Django` 查询语法的工作方式
- 字段名称不能以下划线结尾，原因在于 `Django` 查询语法的工作方式
- `SQL` 保留字，例如 `JOIN`， `WHERE` 或 `SELECT`， 是可以被用作模型字段名称的（但不建议）

5.1.3.2 字段类型

`Django` 的字段类型与 `MySQL` 数据库的对应关系如下：

Django3.2 字段类型	MySQL 数据类型
`AutoField()`	`INT AUTO_INCREMENT PRIMARY KEY`
`BigAutoField()`	`BIGINT AUTO_INCREMENT PRIMARY KEY`
`BooleanField()`	`TINYINT(1)`
`PositiveSmallIntegerField()`	`SMALLINT UNSIGNED`
`SmallIntegerField()`	`SMALLINT`
`PositiveIntegerField()`	`INT UNSIGNED`
`IntegerField()`	`INT`
`BigIntegerField()`	`BIGINT`
`FloatField()`	`DOUBLE`
`DecimalField(max_digits=None, decimal_places=None)`	`DECIMAL`
`CharField(max_length=None)`	`VARCHAR`
`TextField()`	`LONGTEXT`
`DateField(auto_now=False,auto_now_add=Flase)`	`DATE`
`TimeField()`	`TIME`
`DateTimeField(auto_now=False,auto_now_add=False)`	`DATETIME(6)`
`SlugField()`	`VARCHAR(50)`
`FileField(upload_to='')`	`VARCHAR(100)`
`FilePathField()`	`VARCHAR(100)`
`URLField()`	`VARCHAR(200)`
`EmailField()`	`VARCHAR(254)`
`JSONField()`	`JSON`
`BinaryField()`	`LONGBLOB`
`ImageField(upload_to=None)`	`VARCHAR(100)`

5.1.3.3 字段选项

在`Django`中，模型字段可以使用各种选项来控制其行为和属性。以下是一些常用的模型字段选项：

- `Field.primary_key`

该选项用于表示字段是否为主键约束，如：

```
id = models.PositiveSmallIntegerField(primary_key=True)
```

- `Field.unique`

该选项用于表示字段是否为唯一约束，如：

```
username = models.CharField(max_length=50,unique=True)
```

- `max_length`

该选项用于指定字段类型字段允许的最大字符数，如：

```
email = models.EmailField(max_length=50)
```

- `Field.default`

该选项用于指定字段的默认值，如：

```
salary = models.DecimalField(max_digits=8,decimal_places=2,default=0.00)
```

- `Field.null`

该选项用于指定字段是否为空，默认为`False`，如：

```
telephone = models.CharField(max_length=12,null=True)
```

- `Field.db_column`

默认情况下`Django`将使用变量名称作为字段名称，该选项用于指定字段名称，如：

```
faceurl = models.CharField(max_length=50,db_column='avatar')
```

- `Field.choices`

该选项用于指定字段的枚举值，其类型为元组类型，如：

```
YEAR_IN_SCHOOL_CHOICES = [
    ('FR', 'Freshman'),
    ('SO', 'Sophomore'),
    ('JR', 'Junior'),
    ('SR', 'Senior'),
    ('GR', 'Graduate'),
]
shirt_size = models.CharField(max_length=1, choices=SHIRT_SIZES)
```

- `Field.db_index`

该选项用于指定字段是否为索引字段，默认值为`Flase`。如：

```
age = PositiveSmallIntegerField(db_index=True)
```

六、数据迁移

6.1 概述

数据迁移是指在`Django`应用程序中修改模型后，将这些更改应用于数据库的过程。

数据迁移可以使程序员能够轻松的通过修改模型以达到修改数据表结构的目的，而无需手动更改数据表结构。

6.2 实现

确保项目中安装了mysqlclient

```
pip install mysqlclient
```

数据迁移的步骤如下：

第一步：创建迁移文件，其语法结构如下：

```
python manage.py makemigrations [app_label ... ]
```

> 其中`app_label`为要创建迁移文件的应用名称，多个应用名称之间以空格进行分隔

第二步：应用迁移，其语法结构如下：

```
python manage.py migrate
```

注意:

通过模型类创建数据库表时,如果没有定义主键字段,在进行数据库迁移时,会自动创建自增长的主键字段,名称默认为 id

七、配置测试环境

之前我们都是通过页面访问我们的数据以及还要写大量的路由和视图

这里我们介绍一下在django中做测试怎么做

app01.tests.py

```
from django.test import TestCase

import os
import django
import datetime


if __name__ == '__main__':
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'django_views.settings')
    django.setup()
    #下面就可以单个文件的测试

    #必须在这里写这个导入 不然会报错 因为要先启动settings才能找到你的应用
    from app01 import models

    # #增加
    # #第一种
    # #直接指定对象
    # res = models.User.objects.create(name='admin',age=18,create_time='2021-01-
02')
    # #create() 方法返回值就是创建对象的本身
    # print(res.name)
    # print(res.age)
    #
    # #第二种
    # #通过创建对象
    # user_obj = models.User(name='aaa',age=20,create_time=datetime.date(2022, 5,
18))
    # #保存数据到数据库
    # user_obj.save()

    # 修改
    # 第一种修改:批量修改,将所有查询到的数据值全部修改
    # res = models.User.objects.filter(name='admin').update(age=20)
    # print(res)

    #第二种: 查询指定某个对象,在修改对象的属性,然后再保存
    # get() 直接返回查询的对象,如果数据不存在则报错
    # filter()查询的话就是空 不会报错
    # user_obj = models.User.objects.get(id=14)
    # user_obj.age = 22 #修改对象的属性
    # user_obj.save() #保存

    #删除
```

```
#第一种: 批量删除,根据指定字段将所有查询到的数据值全部删除
res = models.User.objects.filter(age=20).delete()

print(res)

# 第二种: 一般是指定ID进行删除,这样可以保证删除的准确性
res = models.User.objects.filter(id=14).delete()
print(res)


# 查询数据 #获取表中所有的数据,返回Queryset对象
res = models.User.objects.all()
#这样也可以查询第一个值 等同于 res[0]
res = models.User.objects.filter(name='admin').first()
print(res)
```

八、作业

8.1 完成本天的案例